

Doc. no. P2631R0
Date: 2022-09-12
Project: Programming Language C++
Audience: LEWG, LWG
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>
Bryce Adelstein Lelbach <brycelelbach@gmail.com>
Jonathan Wakely <cxx@kayari.org>

Publish TS Library Fundamentals 3 Now!

Table of Contents

- [Revision History](#)
 - [R0: Original](#)
- 1 [Abstract](#)
- 2 [Stating the problem](#)
 - 1 [How Much Has Changed?](#)
 - 2 [Why the Urgency?](#)
 - 3 [Do We Have Precedent?](#)
 - 4 [What Do We Risk Losing?](#)
- 3 [Other Directions](#)
 - 1 [Rebase Onto C++23](#)
 - 2 [Merge into C++23](#)
 - 3 [Merge into C++26](#)
 - 4 [Drop the TS Entirely](#)
- 4 [Proposed Motion](#)
- 5 [Acknowledgements](#)
- 6 [References](#)

Revision History

R0: Original

Original version of the paper for the 2022 August mailing.

1 Abstract

The current draft for the TS Library Fundamentals 3 is complete and up to date. We should move to publish before C++23 requires further work for little benefit.

2 Stating the problem

The current TS Library Fundamentals v2 was published after ballot resolution at the 2016 November Issaquah meeting. It is based on the C++14 standard, so will shortly be close to a decade out of date. In the six year between Issaquah and the 2022 November Kona meeting, the Library Working Groups have maintained and evolved this TS; that work has mostly ceased, but the revised TS remains unpublished. We propose moving to

expediently publish the current working draft, and then removing TS Library Fundamentals from our list of active tasks.

2.1 How Much Has Changed?

How is the Current Draft Different to the Current TS? Compared to TS Library Fundamentals v2, we have:

Rebased from the C++14 standard first onto the C++17 standard, and later onto the C++20 standard. This was a non-trivial task updating the document to current library conventions, in particular replacing all *Requires* clauses with *Mandates*, *Constraints*, and *Preconditions* as appropriate, and updating all numbered cross references into the base standard. Additionally, we consistently switched from typedef to using for type aliases, and use nested namespace declarations to simplify header synopses etc.

Added one significant new experimental library (with a full implementation now checked into gcc libstdc++ trunk):

- `scope_exit`
- `scope_fail`
- `scope_success`
- `unique_resource`

Removed the features that landed in the base standard, either C++17 or C++20:

- conjunction, disjunction, negation
- gcd, lowest common multiple
- polymorphic memory resources
- searchers
- `shared_ptr` support for arrays
- source location
- `std::any`
- `std::apply`
- `std::array` creation functions
- `std::not_fn`
- `std::optional`
- `std::string_view`
- type trait variable templates
- uniform container erasure

Resolved a small set of issues filed over the 6 year development period:

- [#2960](#) `nonesuch` is insufficiently useless
- [#3134](#) LFTSv3 contains extraneous [meta] variable templates that should have been deleted by P09961
- [#3135](#) LFTSv3 contains two redundant alias templates
- [#3413](#) `propagate_const`'s `swap`'s `noexcept` specification needs to be constrained and use a trait
- [#3649](#) Reinstate and bump `__cpp_lib_experimental_memory_resource` feature test macro

2.2 Why the Urgency?

The currently published TS Library Fundamentals 2 is based on C++14, and will soon be close to a decade out of date with respect to the standard it is based on. Many features of that TS are now adopted into the base standard itself, and the words we use to describe library features changed significantly in C++20.

The current working draft for TS Library Fundamentals 3 has been rebased (twice) and is currently drafted relative to C++20. It is correctly worded in the current form, and has removed all features that have landed in the

C++17 and C++20 standards. It is a much more stable document to maintain.

If we send the TS to ballot at the Kona meeting, and get a good turnaround on the timetable, our following meeting could be ballot resolution. If we can turn that around in a single meeting, then the protocol is that there are no further ISO ballots, and we can proceed straight to publishing. Meanwhile, we expect both the Kona meeting and the following meeting to also be ballot resolution meetings for the base C++23 standard itself. The net effect, if we keep to our planned timetable, as that C++23 would go off for final DIS ballot while the TS Library Fundamentals 3 proceeds straight to publication, still based on the current standard, C++20.

If we do not immediately ballot the TS as-is, we will inevitably face more work to rebase the TS (a third time) onto C++23, and with the current workload scheduled for the Library Working Group, that is an exercise we would prefer to avoid, even though the relative changes are expected to be minor.

2.3 Do We Have Precedent?

For reference, here is how the TS Library Fundamentals v2 went through ISO balloting, concurrently with C++17. Remember, that TS was based on C++14, so also had to publish before the new base standard:

LFTS PDTS ballot	October	2015, Kona
C++17 CD ballot,	June	2016, Oulu
LFTS Ballot resolution and publish	November	2016, Issaquah
C++17 DIS ballot	February	2017, Kona
C++17 Published	November	2017

Here is the corresponding timetable we propose for TS Library Fundamentals v3:

C++23 CD ballot,	July	2022, Telecon
LFTS DTS ballot	November	2022, Kona
LFTS Ballot resolution and publish	Spring	2023, TBD
C++23 DIS ballot to publish	Spring	2023, TBD
C++23 Published	November?	2023

Note that our Convener informs me that PDTS is no longer a thing, and the actual ballot would be a DTS ballot, as reflected in the timetable above.

Observe that the order of the initial CD ballot for the base standard and the (P)DTS ballot have been reversed. This timetable relies on the convener being able to complete the PDTS ballot before the 2023 Spring meeting, which is tight. Finally, if all goes well, the PDTS and C++23 complete ballot resolution at the same meeting. The key difference is that the DTS can proceed direct to publication, while C++20 remains the current standard, while C++23 must pass another round of balloting for the DIS. If any of these deadlines slip, we will most likely have to rebase onto C++23, likely delaying publication by a year or more.

2.4 What Do We Risk Losing?

Nothing! Releasing now, rather than waiting to accumulate more features and changes, essentially costs nothing as there are no expected features and changes that we are waiting for. The paper queue is empty, and the draft is fully updated for the current C++ standard.

TS Library Fundamentals 3 would be a smaller release than the previous two editions of the TS, but that is in keeping with a change in Library Evolution Working Group practice, now that we are on a regular 3 year release cycle, which is to more eagerly send features to the base standard itself. The Library Working Group is already oversupplied with papers to consider for the base standard, so further TS work would be relegated to such an extent that we would be ready to release TS Library Fundamentals v4 by the time further work completed, assuming that we send TS Library Fundamentals v3 to ballot at the next meeting.

3 Other Directions

Several other approaches were considered and rejected by the authors. Here we record those ideas and the reasoning by which they were set aside.

3.1 Rebase Onto C++23

Rather than race the C++23 standard into print, we could simply wait for it to ship, rebase the current TS draft onto C++23, and then go to ballot in 2024 once the revised standard is published.

This direction has a lot to commend it, as the end result will be stable and meaningful for longer. The downside is that this involves further LWG work to review the rebase for correctness, and leaves the whole project hanging around for another 12-18 months with no actual progress expected.

We reject this direction mostly for the desire to efficiently complete the outstanding work and then retire the project, reducing the task list hanging over the Library Working Group.

3.2 Merge into C++23

It is simply way too late in the process to consider adding new material to C++23, so this idea fails on the timetable, before even considering its merits.

3.3 Merge into C++26

We might instead skip publishing the TS v3, and directly merge the material into the working draft for C++26 when it opens. This remains a possibility even if we do publish the TS promptly though, and discounts the work necessary to review the features for adoption into the base standard. There are reasons that these features are currently held in a TS, and were not immediately adopted by earlier standards, even if the choice of TS vs Standard might be different if presented anew today. Some of these features have been evaluated for integration previously, and deemed to need more work, such as the stream joiners. Meanwhile, the published TS v2 gets more and more out of date as the base standard continues to evolve.

We reject this direction only in the sense that we reject it is a good reason to defer publishing the current draft as the new TS. We expect part of the Library Evolution process for C++26 will be to review and incorporate the contents of this TS, as appropriate.

3.4 Drop the TS Entirely

Another idea is that with the LWG backlog of proposals for the base standard, and the standard shipping on a 3 year cadence, there is no time for experimental work such as a TS. As such, we should simply abandon the project.

This suggestion is rejected by the authors, as we have already registered a work item with ISO and are official in stage 20.00, Preparatory: New project registered in TC/SC work programme, and it would be embarrassing for future requests if we abandon this project uncompleted, like the Arrays TS. Further, the existing work and time invested in the current TS draft is ready to ballot, and it would be a shame to lose our sunk cost.

However, it is recognised that this will likely be the last such TS for the foreseeable future, and we should acknowledge that.

4 Proposed Motion

The following is based on the motion for TS Library Fundamentals 2 at the 2015 Kona meeting, reworded as we have already created a New Work Item and approved the working paper at 2018 Jacksonville meeting.

Move to direct the Convener to appoint an editing committee composed of *reviewer 1*, *reviewer 2*, and *reviewer 3* to approve the correctness of the working paper [N4920](#) as amended by the straw polls approved at this meeting, and to direct the Convener to transmit the approved updated working paper for DTS ballot.

5 Acknowledgements

Thanks to all the LEWG and LWG participants who have kept this work alive over the last 6 years, and special thanks to Thomas Köppe for his proactive work as the project editor, the main reason we are in such good shape that we can confidently propose going to ballot on barely a meeting's notice!

6 References

- [N4908](#) Working Draft, C++ Extensions for Library Fundamentals, Version 3, Thomas Köppe
- [P2081R1](#) Rebase the Library Fundamentals v3 TS on C++20, Thomas Köppe
- [P0996R1](#) Rebase Library Fundamentals TS on C++17, Alisdair Meredith
- [P1210R0](#) Completing the Rebase of Library Fundamentals, Version 3, Working Draft, Barry Hedquist
- [P0052R10](#) Generic Scope Guard and RAII Wrapper for the Standard Library, Peter Sommerlad, Andrew L. Sandoval